

UPES DEHRADUN

EXAMINATION ANSWER BOOKLET

(Candidate Details Page – binds identity to the entire booklet)



Exam ID:

CSEG 1032

Subject:

Computer Organization and Architecture

Date:

25-10-25

SAP ID:

590013715

Enrol No:

240010178913

Course / Programme:

B-TECH (CSE)

Year / Semester:

2

Batch:

27

Instructions:

- Fill in all details clearly before starting.
- Write the Question Number in the boxed area on each page.
- Do not write in or over the QR code area.
- Use extra sheets only if required; attach them at the end.
- On your last used page, fill total pages and sign.

Total pages used (incl. extras):

Invigilator Signature:

Date:

Candidate Signature:

Date:



Arithmetic Micro Operations

- i) Perform basic arithmetic operations on binary data.
- ii) Addition, Subtraction, increment, decrement, Complement.
- iii) Changes numeric value of data
- iv) ALU performs arithmetic operations
- v) May generate carry or overflow flags.

Shift Micro Operations

- i) Move bits left or right in a register.
- ii) Logical Shift, arithmetic shift, Circular Shift.
- iii) Changes bit positions but not the value (except sign).
- iv) Shifter circuit performs shifting.
- v) No carry or overflow generated.



Hardwired Control

- i) Generated by combinational logic
- ii) It is faster.
- iii) Difficult to modify
- iv) Complex and costly hardware.
- v) RISC Systems

Microprogrammed Control

- i) Stored in control memory (firmware)
- ii) Slower due to memory access.
- iii) Easy to modify or update.
- iv) Simple, cheaper hardware.
- v) CISC Systems.

Q. No

3



DMA (Digital Memory Arithmetic) controls cache memory and performs logical AND/OR operations in CPU. The controller decodes instructions and handles stack underflow and overflow.

Q. No

4



Horizontal Micro instructions

- i) Wide word; each bit controls one signal.
- ii) Fast (parallel control)
- iii) Highly flexible
- iv) Large
- v) High-speed processors

Vertical Micro Instructions

- i) Compact word; encoded fields.
- ii) Slower (needs decoding)
- iii) Less flexible
- iv) Small
- v) Low-cost control units



Programmed I/O

- i) CPU continuously checks device status (polling)
- ii) Inefficient - CPU waits for I/O completion.
- iii) Controlled by software loop.
- iv) ~~used~~ Not used
- v) Simple microcontrollers

Interrupt-Driven I/O

- i) CPU executes program until device sends interrupt.
- ii) Efficient - CPU used for other tasks.
- iii) Controlled by hardware interrupt signals.
- iv) Used to signal completion.
- v) Modern I/O systems.



Stored Program Organization:-

- i) The stored program concept was introduced by John von Neumann.
- ii) Both instructions and data are stored in the same memory unit.
- iii) The CPU fetches an instruction from memory, decodes it and executes it sequentially unless a control transfer occurs.
- iv) The memory is accessed by an address register (MAR) and the data is held temporarily in the data register (MDR).
- v) The program counter (PC) keeps track of the next instruction address.

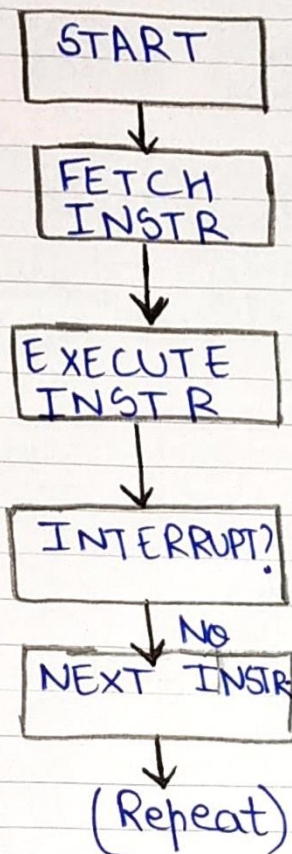
4096 × 16 Memory Organization

Total memory words = 4096 = 2^{12} , hence 12 address lines

Each word = 16 bits

A typical instruction format can be

Bits	Field	Description
15-12	opcode	Specifies operation
11-0	Address	Specifies memory operand





- i) Single Accumulator Organization - One Register (Acc) holds intermediate results; Simple hardware.
- ii) General Register Organization - Multiple registers; instructions specify operands explicitly.
- iii) Stack Organization - operands stored on stack; ALU operates on TOS (top of stack).

Exam ID: MATH101-2025

Q. No

8

Page: 10



Type	Description	Example
SSI SD	One instruction, one data stream	Uni processor
SIMD	One instruction, multiple data stream	GPUs, Vector Processors
MISD	Multiple instruction, single data	Fault-Tolerant System
MIMD	Multiple instruction, multiple data	Multicore processor



i) One-Address Instruction (Accumulator Based)

In a one-address machine, operations use a single memory address and an implicit accumulator register. Data from memory is loaded into the accumulator, the operation is performed, and the result is stored back.

Instructions:

LOAD A : $AC = A$
 ADD B : $AC = AC + B$
 STORE TEMP1: $TEMP1 = A + B$
 LOAD C : $AC = C$
 ADD D : $AC = AC + D$
 STORE TEMP2: $TEMP2 = C + D$
 LOAD TEMP1: $AC = TEMP1$
 MUL TEMP2: $AC = TEMP1 * TEMP2$
 STORE X : $X = (A + B) * (C + D)$

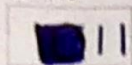
opcode	operand -s of operand	address	mode

ii) Two-Address Instruction:-

Two-address instructions specify two operands. The result is stored in one of the operands.

MOV TEMP1, A ; $TEMP1 = A$
 ADD TEMP1, B ; $TEMP1 = TEMP1 + B \Rightarrow$

Q. No



$TEMP1 = A + B$
 $MOV\ TEMP2,\ C \quad ;\ TEMP2 = C$
 $ADD\ TEMP2,\ D \quad ;\ TEMP2 = TEMP2 + D = >$
 $TEMP2 = C + D$
 $MUL\ TEMP1,\ TEMP2 \quad ;\ TEMP1 = TEMP1 * TEMP2$
 $=> TEMP1 = (A+B) * (C+D)$
 $MOV\ X,\ TEMP1 \quad ;\ X = TEMP1$

iii) Three Address Instruction

Three-address instructions specify two source operands and one destination operand. This allows direct computation without temporary storage.

$ADD\ TEMP1,\ A,\ B \quad ;\ TEMP1 = A + B$
 $ADD\ TEMP2,\ C,\ D \quad ;\ TEMP2 = C + D$
 $MUL\ X,\ TEMP1,\ TEMP2 \quad ;\ X = TEMP1 * TEMP2 =>$
 $X = (A+B) * (C+D)$



iv) Zero-Address Instruction:-

Zero-address machines use a stack for computations. Operands are pushed onto the stack, and operators pop operands and push results.

PUSH A
PUSH B
A DD

; $AC = A + B$, result on top of stack

PUSH C
PUSH D
ADD

; $AC = C + D$, result on top of stack

MUL

; $AC = (A + B) * (C + D)$, result on stack

POP X

; $X = \text{result}$.

v) RISC Instruction

RISC (Reduced Instruction Set Computer) uses Load/Store architecture. Arithmetic instructions operate only on registers, not directly on memory. Memory is accessed via LOAD/STORE.



Instructions :-

LOAD R1, A	; R1 = A
LOAD R2, B	; R2 = B
ADD R3, R1, R2	; R3 = A + B
LOAD R4, C	; R4 = C
LOAD R5, D	; R5 = D
ADD R6, R4, R5	; R6 = C + D
MUL R7, R3, R6	; R7 = (A + B) * (C + D)
STORE X, R7	; X = (A + B) * (C + D)
	&



Data transfer between an I/O device and CPU can occur through programmed I/O or interrupt-driven I/O. Here, we illustrate a general procedure using a control and status register.

→ Procedure for Setting and Clearing Flag bit.

- Setting flag - When the I/O device is ready, the device sets a status flag bit (e.g., $READY=1$).
- Clearing bit - After CPU reads data, the flag is cleared ($READY=0$).

→ Data transfer from I/O Device to CPU

